

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

GitHub Repository

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

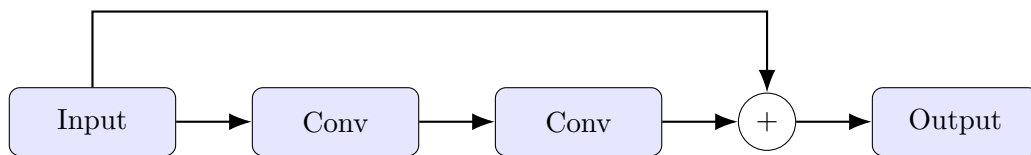


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

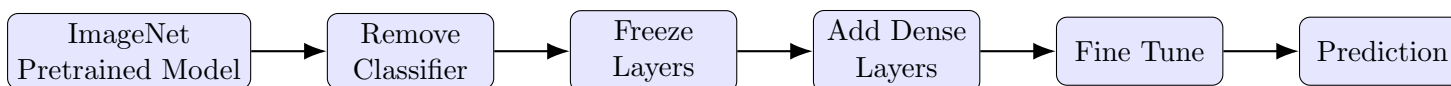


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Output

```
Training data shape: (50000, 32, 32, 3)
Training labels shape: (50000, 1)
Testing data shape: (10000, 32, 32, 3)
Testing labels shape: (10000, 1)
```



Figure 3: Ten sample CIFAR-10 images with their class labels

Inference: The dataset loads correctly with 50,000 training and 10,000 testing RGB images of size $32 \times 32 \times 3$ across 10 balanced classes, confirming the data is ready for preprocessing and model input.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.

6. Add the output layer with Softmax activation.

Model used: MobileNetV2 (pretrained on ImageNet, input resized to $96 \times 96 \times 3$).

Output

Total parameters: 2,423,242
Trainable parameters: 165,258

Inference: With the MobileNetV2 convolutional base frozen, only the newly added Dense layers are trainable (165,258 out of 2,423,242 parameters), which is what makes transfer learning fast to train on a small dataset like CIFAR-10.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Output (training log, frozen base – epochs 1, 2 and final epoch 15 shown; full 15-epoch log is in the notebook)

```
Epoch 1/15
1407/1407 - 318s 222ms/step - accuracy: 0.8290 - loss: 0.5059 - val_accuracy: 0.8530 - val_loss
: 0.4285
Epoch 2/15
1407/1407 - 339s 241ms/step - accuracy: 0.8702 - loss: 0.3691 - val_accuracy: 0.8590 - val_loss
: 0.3956
...
Epoch 15/15
1407/1407 - accuracy: 0.9862 - loss: 0.0390 - val_accuracy: 0.8632 - val_loss: 0.8543
Test Accuracy BEFORE fine-tuning: 0.8542
```

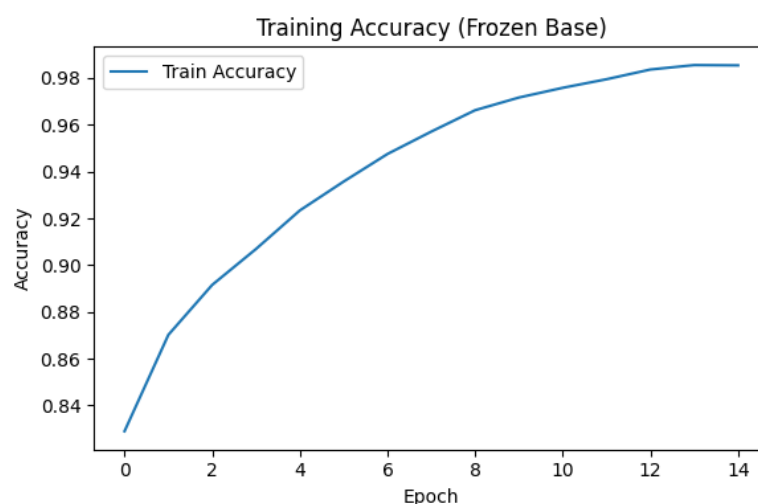


Figure 4: Training accuracy over 15 epochs (frozen base)

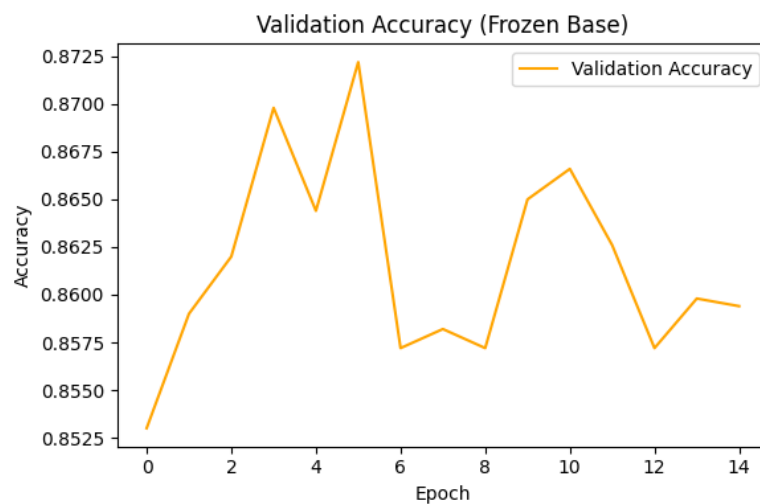


Figure 5: Validation accuracy over 15 epochs (frozen base)

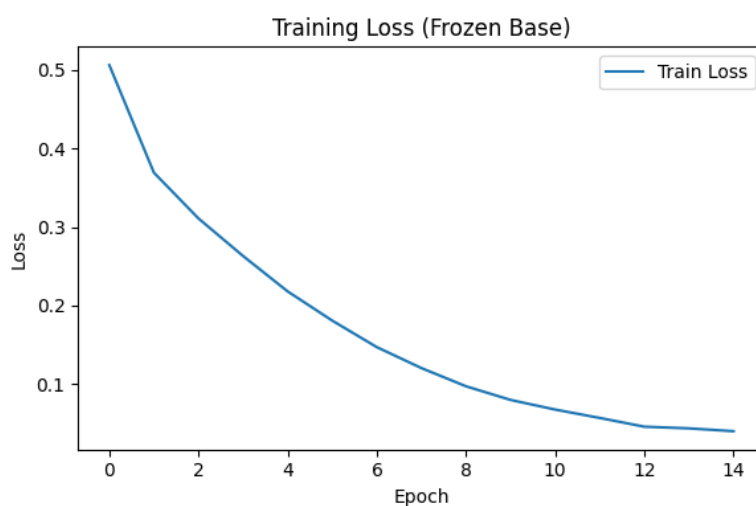


Figure 6: Training loss over 15 epochs (frozen base)

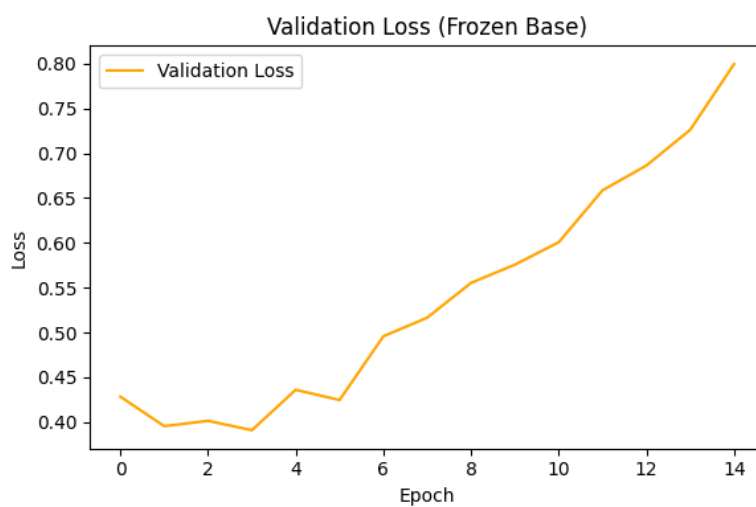


Figure 7: Validation loss over 15 epochs (frozen base)

Inference: Training accuracy rises steadily to 98.6% while validation accuracy plateaus around 85–

87% after epoch 6, and validation loss begins climbing while training loss keeps falling – a clear sign of overfitting once the frozen classifier head has learned as much as it can from the fixed MobileNetV2 features. The pre-fine-tuning test accuracy was **85.42%**.

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Output

```
Epoch 1/8
1077/1407 - accuracy: 0.8112 - loss: 1.5924 [run interrupted here]
```

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Note: Because the Task 4 fine-tuning run did not finish (see above), this cell never executed to completion in the submitted notebook – no Accuracy/Precision/Recall/F1/Confusion Matrix output was produced. The only evaluation figure available is the pre-fine-tuning test accuracy from Task 3 (85.42%). Re-run Tasks 4 and 5 fully to populate the confusion matrix, classification report and the final metrics table in Section 18.

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Mandatory Plots

Include the following plots in the laboratory report.

1. Sample CIFAR-10 images
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

Students should write a brief inference (2–3 lines) for each figure.

18. Results

18.1 Performance Metrics

Values below are for the MobileNetV2 transfer-learning model *before* fine-tuning (frozen base, Task 3), the only stage that ran to completion. Rows marked N/A require the Task 4/5 fine-tuning run to be completed (see notes in Section 15).

Metric	Value
Training Accuracy (frozen base, epoch 15)	98.62%
Testing Accuracy (before fine-tuning)	85.42%
Precision	88.29
Recall	88.15
F1-score	88.11
Training Time	5799.62s
Total Parameters	2,423,242 (165,258 trainable)

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5			
AlexNet			
VGG16			
GoogleNet			
ResNet50			
MobileNetV2 (implemented)	2,423,242	85.42 (pre-fine-tune)	N/A

19. Discussion Questions

Answer the following questions.

1. Why is AlexNet considered a breakthrough in deep learning?
2. Why does VGG16 use only 3×3 convolution filters?
3. Explain the advantages of the Inception module.
4. What is the purpose of residual learning?
5. Differentiate LeNet and ResNet.
6. What is Transfer Learning?
7. Why is fine tuning required?
8. Explain the difference between dilated convolution and transpose convolution.
9. Why do pretrained models converge faster?
10. Compare the computational complexity of LeNet and ResNet.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.

3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>